

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

старший преподаватель

должность, уч. степень, звание

подпись, дата

Д.О. Шевяков

инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №2

СОЗДАНИЕ ПРИЛОЖЕНИЯ ИНТЕРНЕТА ВЕЩЕЙ И ВЕБ-ИНТЕРФЕЙСА

по курсу: Интернет вещей

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № _____ 4326

подпись, дата

Г. С. Томчук

инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Приобретение навыков создания и запуска приложения интернета вещей.

2 Задание

Создать приложение на Flask. Реализовать классы и их иерархию, согласно сданной первой лабораторной работе, показать работу методов этих классов. (Не нужно писать предполагаемый метод целиком, на данном этапе достаточно вывести в лог сервера сообщение о том, что метод был запущен) .

Вариант задания - Умная теплица.

3 Выполнение задания

В отличие от первой работы, где система проектировалась на уровне диаграмм классов, здесь требуется реализовать спроектированную иерархию классов на языке Python с помощью Flask и интегрировать её в веб-приложение, чтобы наглядно продемонстрировать работоспособность методов через вывод сообщений в лог сервера.

В процессе выполнения работы была создана программная модель умной теплицы, полностью соответствующая ранее разработанной UML-диаграмме. Для реализации абстрактного класса Device был использован модуль abc стандартной библиотеки Python. Класс Device унаследован от ABC, а его методы turn_on, turn_off и get_status объявлены с реализацией, поскольку они являются общими для всех устройств (рисунок 1).

```
class Device(ABC):
    def __init__(self, device_id: int, name: str, status: bool = False):
        self.id = device_id
        self.name = name
        self.status = status

    def turn_on(self):
        self.status = True
        print(f"[LOG] {self.__class__.__name__} {self.name} (ID: {self.id}) включён")

    def turn_off(self):
        self.status = False
        print(f"[LOG] {self.__class__.__name__} {self.name} (ID: {self.id}) выключен")

    def get_status(self) -> bool:
        print(f"[LOG] Запрос статуса {self.__class__.__name__} {self.name} (ID: {self.id}) -> {self.status}")
        return self.status
```

Рисунок 1 – Абстрактный класс Device

От него были унаследованы конкретные классы Sensor и Actuator. В классе Sensor добавились атрибуты для хранения типа датчика, единиц измерения и текущего значения, а также методы measure для имитации измерения и calibrate для калибровки (рисунок 2).

```

class Sensor(Device):
    def __init__(self, device_id: int, name: str, sensor_type: str, unit: str):
        super().__init__(device_id, name)
        self.type = sensor_type
        self.unit = unit
        self.value: float = 0.0

    def measure(self):
        self.value = 22.5
        print(f"[LOG] Датчик {self.name} (ID: {self.id}) измерил: {self.value} {self.unit}")

    def calibrate(self, value: float):
        self.value = value
        print(f"[LOG] Датчик {self.name} (ID: {self.id}) откалиброван на значение {value} {self.unit}")

```

Рисунок 2 – Класс Sensor

В классе Actuator были введены атрибуты типа исполнительного устройства и мощности, а также методы `apply_action` для выполнения команд и `set_power` для изменения мощности (рисунок 3).

```

class Actuator(Device):
    def __init__(self, device_id: int, name: str, actuator_type: str, power: float = 0.0):
        super().__init__(device_id, name)
        self.type = actuator_type
        self.power = power

    def apply_action(self, action: str):
        print(f"[LOG] Исполнительное устройство {self.name} (ID: {self.id}) выполняет действие: {action}")

    def set_power(self, level: float):
        self.power = level
        print(f"[LOG] Мощность устройства {self.name} (ID: {self.id}) установлена на {level} Вт")

```

Рисунок 3 – Класс Actuator

Дополнительно были реализованы классы `MainControlUnit` и `Database`. Первый отвечает за логику управления: обработку данных с датчиков, отправку команд актуаторам и проверку тревог (рисунок 4).

```

class MainControlUnit:
    def __init__(self):
        self.mode = "auto"
        self.schedule: Dict[str, str] = {}
        self.alerts: List[str] = []

    def process_data(self, sensor_data: Dict[int, float]):
        print(f"[LOG] Контроллер обрабатывает данные с датчиков: {sensor_data}")

    def send_command(self, actuator: Actuator, command: str):
        print(f"[LOG] Контроллер отправляет команду '{command}' устройству {actuator.name} (ID: {actuator.id})")
        actuator.apply_action(command)

    def check_alerts(self):
        print("[LOG] Контроллер проверяет наличие тревог...")
        if self.alerts:
            print(f"[LOG] Активные тревоги: {self.alerts}")
        else:
            print("[LOG] Тревог нет")

```

Рисунок 4 – Класс MainControlUnit

Второй имитирует работу базы данных: сохранение показаний и событий, а также получение истории (рисунок 5).

```
class Database:
    def __init__(self, connection_string: str, storage_path: str):
        self.connection_string = connection_string
        self.storage_path = storage_path

    def save_reading(self, sensor_id: int, value: float, timestamp: datetime):
        print(f"[LOG] БД: сохранено показание датчика {sensor_id} = {value} в {timestamp}")

    def save_event(self, device_id: int, action: str, timestamp: datetime):
        print(f"[LOG] БД: сохранено событие устройства {device_id}: {action} в {timestamp}")

    def get_history(self, start: datetime, end: datetime, device_id: Optional[int] = None):
        print(f"[LOG] БД: запрос истории с {start} по {end} для устройства {device_id if device_id else 'всех'}")

        return []
```

Рисунок 5 – Класс Database

Интеграция классов с веб-приложением была выполнена с помощью фреймворка Flask.

В файле app.py был создан экземпляр приложения Flask и определены два маршрута: корневой маршрут “/” и дополнительный “/sensor”. При обращении к корневому маршруту создаются экземпляры всех основных классов: датчика температуры, вентилятора, контроллера и базы данных. Затем последовательно вызываются методы этих классов: включение датчика, измерение, калибровка, выключение; включение вентилятора, выполнение действия, изменение мощности, выключение, обработка данных контроллером, проверка тревог, отправка команды, сохранение показания и события в базу данных, запрос истории (рисунок 6).

```

@app.route('/')
def index():
    # Датчик температуры
    temp_sensor = Sensor(device_id: 1, name: "Датчик температуры", sensor_type: "temperature", unit: "C")
    temp_sensor.turn_on()
    temp_sensor.measure()
    temp_sensor.calibrate(23.0)
    status = temp_sensor.get_status()
    temp_sensor.turn_off()

    # Исполнительное устройство (вентилятор)
    fan = Actuator(device_id: 101, name: "Вентилятор", actuator_type: "fan", power: 150)
    fan.turn_on()
    fan.apply_action("увеличить обороты")
    fan.set_power(200)
    fan.turn_off()

    # Контроллер
    controller = MainControlUnit()
    controller.process_data({1: 23.0, 2: 65.0})
    controller.check_alerts()
    controller.send_command(fan, command: "включить на полную мощность")

    # База данных
    db = Database(connection_string: "sqlite:///greenhouse.db", storage_path: "./data")
    now = datetime.now()
    db.save_reading(sensor_id: 1, value: 23.0, now)
    db.save_event(device_id: 101, action: "вентилятор включён", now)
    history = db.get_history(now, now)

    return "Проверьте терминал. В нём отражены логи сервера."

```

Рисунок 6 – Корневой маршрут

Второй маршрут демонстрирует аналогичную работу с датчиком влажности. Все вызовы методов сопровождаются выводом в консоль (лог сервера) сообщений, начинающихся с [LOG], что позволяет наглядно проследить последовательность выполнения. В браузер же возвращается текст, подтверждающий, что обработка запроса завершена (рисунок 7).

```

@app.route('/sensor')
def sensor_example():
    humidity = Sensor(device_id: 2, name: "Датчик влажности", sensor_type: "humidity", unit: "%")
    humidity.turn_on()
    humidity.measure()
    humidity.calibrate(65.0)
    humidity.turn_off()
    return "Датчик влажности сработал. Смотрите лог."

```

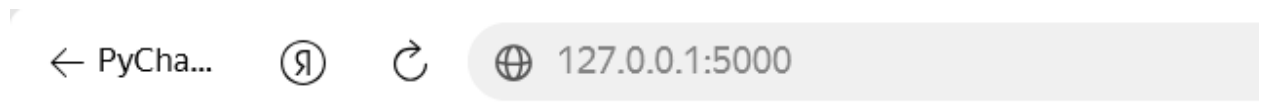
Рисунок 7 – Дополнительный маршрут

При запуске приложения app.py в терминале появляется сообщение о запуске сервера (рисунок 8).

```
C:\Users\admin\PycharmProjects\smart_greenhouse\.venv\Scripts\python.exe C:\Users\admin\PycharmProjects\smart_greenhouse\.venv\app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 121-496-237
```

Рисунок 8 – Запуск сервера

При переходе в браузере по адресу <http://127.0.0.1:5000/> пользователь видит текстовую строку, а в терминале отображаются все сообщения от вызванных методов (рисунки 9-10).



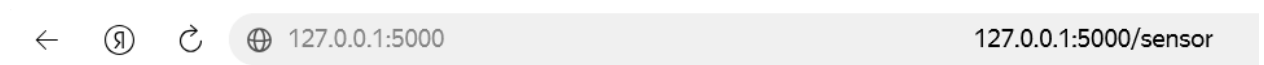
Проверьте терминал. В нём отражены логи сервера.

Рисунок 9 – Веб-страница

```
[LOG] Sensor Датчик температуры (ID: 1) включён
[LOG] Датчик Датчик температуры (ID: 1) измерил: 22.5 C
[LOG] Датчик Датчик температуры (ID: 1) откалиброван на значение 23.0 C
[LOG] Запрос статуса Sensor Датчик температуры (ID: 1) -> True
[LOG] Sensor Датчик температуры (ID: 1) выключен
[LOG] Actuator Вентилятор (ID: 101) включён
[LOG] Исполнительное устройство Вентилятор (ID: 101) выполняет действие: увеличить обороты
[LOG] Мощность устройства Вентилятор (ID: 101) установлена на 200 Вт
[LOG] Actuator Вентилятор (ID: 101) выключен
[LOG] Контроллер обрабатывает данные с датчиков: {1: 23.0, 2: 65.0}
[LOG] Контроллер проверяет наличие тревог...
[LOG] Тревог нет
[LOG] Контроллер отправляет команду 'включить на полную мощность' устройству Вентилятор (ID: 101)
[LOG] Исполнительное устройство Вентилятор (ID: 101) выполняет действие: включить на полную мощность
[LOG] БД: сохранено показание датчика 1 = 23.0 в 2026-03-22 13:18:41.474954
[LOG] БД: сохранено событие устройства 101: вентилятор включён в 2026-03-22 13:18:41.474954
[LOG] БД: запрос истории с 2026-03-22 13:18:41.474954 по 2026-03-22 13:18:41.474954 для устройства всех
```

Рисунок 10 – Логи сервера основного маршрута

То же самое было сделано для дополнительного маршрута (рисунки 11-12).



Датчик влажности сработал. Смотрите лог.

Рисунок 11 – Веб-страница

```
127.0.0.1 - - [22/Mar/2026 13:18:41] "GET / HTTP/1.1" 200 -  
[LOG] Sensor Датчик влажности (ID: 2) включён  
[LOG] Датчик Датчик влажности (ID: 2) измерил: 22.5 %  
[LOG] Датчик Датчик влажности (ID: 2) откалиброван на значение 65.0 %  
[LOG] Sensor Датчик влажности (ID: 2) выключен
```

Рисунок 12 – Логи сервера дополнительного маршрута

Листинг программы представлен в Приложениях к лабораторной работе.

Вывод

В ходе выполнения второй лабораторной работы приобретены навыки создания и запуска приложения интернета вещей с использованием фреймворка Flask. Разработанная диаграмма классов была успешно реализована на языке Python. Спроектирована иерархия классов, включающая абстрактный класс Device, наследующие от него классы Sensor и Actuator, а также классы MainControlUnit и Database. Все методы классов, в соответствии с заданием, были реализованы с выводом информационных сообщений в консоль, что позволяет наглядно проследить последовательность их вызова и корректность взаимодействия между компонентами системы.

Интеграция классов с веб-приложением была выполнена с помощью Flask. Создано приложение, которое при обращении к корневому маршруту и дополнительному маршруту создаёт объекты спроектированных классов и вызывает их методы, имитируя реальный процесс управления устройствами, обработки информации и сохранения истории.

Таким образом, в процессе выполнения лабораторной работы были закреплены навыки объектно-ориентированного программирования на Python, включая работу с абстрактными классами и наследованием, освоены основы создания веб-приложений с использованием фреймворка Flask, а также получен практический опыт интеграции модели классов с веб.

ПРИЛОЖЕНИЯ

ПРИЛОЖЕНИЕ А

Листинг `devices.py`

```
from abc import ABC, abstractmethod
from datetime import datetime
from typing import Dict, List, Optional, Union

class Device(ABC):
    def __init__(self, device_id: int, name: str, status: bool = False):
        self.id = device_id
        self.name = name
        self.status = status

    def turn_on(self):
        self.status = True
        print(f'[LOG] {self.__class__.__name__} {self.name} (ID: {self.id}) включён')

    def turn_off(self):
        self.status = False
        print(f'[LOG] {self.__class__.__name__} {self.name} (ID: {self.id}) выключен')

    def get_status(self) -> bool:
        print(f'[LOG] Запрос статуса {self.__class__.__name__} {self.name} (ID: {self.id}) -> {self.status}')
        return self.status

class Sensor(Device):
    def __init__(self, device_id: int, name: str, sensor_type: str, unit: str):
        super().__init__(device_id, name)
        self.type = sensor_type
        self.unit = unit
        self.value: float = 0.0
```

```

def measure(self):
    self.value = 22.5
    print(f"[LOG] Датчик {self.name} (ID: {self.id}) измерил: {self.value} {self.unit}")

def calibrate(self, value: float):
    self.value = value
    print(f"[LOG] Датчик {self.name} (ID: {self.id}) откалиброван на значение {value} {self.unit}")

class Actuator(Device):
    def __init__(self, device_id: int, name: str, actuator_type: str, power: float = 0.0):
        super().__init__(device_id, name)
        self.type = actuator_type
        self.power = power

    def apply_action(self, action: str):
        print(f"[LOG] Исполнительное устройство {self.name} (ID: {self.id}) выполняет действие: {action}")

    def set_power(self, level: float):
        self.power = level
        print(f"[LOG] Мощность устройства {self.name} (ID: {self.id}) установлена на {level} Вт")

class MainControlUnit:
    def __init__(self):
        self.mode = "auto"
        self.schedule: Dict[str, str] = {}
        self.alerts: List[str] = []

    def process_data(self, sensor_data: Dict[int, float]):
        print(f"[LOG] Контроллер обрабатывает данные с датчиков: {sensor_data}")

```

```

def send_command(self, actuator: Actuator, command: str):
    print(f"[LOG] Контроллер отправляет команду '{command}' устройству {actuator.name} (ID: {actuator.id})")
    actuator.apply_action(command)

def check_alerts(self):
    print("[LOG] Контроллер проверяет наличие тревог...")
    if self.alerts:
        print(f"[LOG] Активные тревоги: {self.alerts}")
    else:
        print("[LOG] Тревог нет")

class Database:
    def __init__(self, connection_string: str, storage_path: str):
        self.connection_string = connection_string
        self.storage_path = storage_path

    def save_reading(self, sensor_id: int, value: float, timestamp: datetime):
        print(f"[LOG] БД: сохранено показание датчика {sensor_id} = {value} в {timestamp}")

    def save_event(self, device_id: int, action: str, timestamp: datetime):
        print(f"[LOG] БД: сохранено событие устройства {device_id}: {action} в {timestamp}")

    def get_history(self, start: datetime, end: datetime, device_id: Optional[int] = None):
        print(f"[LOG] БД: запрос истории с {start} по {end} для устройства {device_id if device_id else 'всех'}")
        return []

```

ПРИЛОЖЕНИЕ В

Листинг app.py

```
from flask import Flask
from datetime import datetime
from devices import Sensor, Actuator, MainControlUnit, Database

app = Flask(__name__)

@app.route('/')
def index():
    # Датчик температуры
    temp_sensor = Sensor(1, "Датчик температуры", "temperature", "C")
    temp_sensor.turn_on()
    temp_sensor.measure()
    temp_sensor.calibrate(23.0)
    status = temp_sensor.get_status()
    temp_sensor.turn_off()

    # Исполнительное устройство (вентилятор)
    fan = Actuator(101, "Вентилятор", "fan", 150)
    fan.turn_on()
    fan.apply_action("увеличить обороты")
    fan.set_power(200)
    fan.turn_off()

    # Контроллер
    controller = MainControlUnit()
    controller.process_data({1: 23.0, 2: 65.0})
    controller.check_alerts()
    controller.send_command(fan, "включить на полную мощность")

    # База данных
    db = Database("sqlite:///greenhouse.db", "./data")
    now = datetime.now()
```

```
db.save_reading(1, 23.0, now)
db.save_event(101, "вентилятор включён", now)
history = db.get_history(now, now)

return "Проверьте терминал. В нём отражены логи сервера."
```

```
@app.route('/sensor')
def sensor_example():
    humidity = Sensor(2, "Датчик влажности", "humidity", "%")
    humidity.turn_on()
    humidity.measure()
    humidity.calibrate(65.0)
    humidity.turn_off()
    return "Датчик влажности сработал. Смотрите лог."
```

```
if __name__ == '__main__':
    app.run(debug=True)
```